

Итоговый отчёт по код-ревью

Это отчёт по ревью, которое я провёл для PR #1 «added lesson 8 initial code» в репозитории `slavacalestru/mlops-sprint2`, ветка `feature/ml-pipeline` в `main`. Всего я оставил в PR 30 комментариев к строкам кода и два общих комментария, а здесь собрал главное.

Что я проверял

В PR лежит небольшой ML-пайплайн, который предсказывает стоимость поездки такси по датасету `uber.csv` на 200 тысяч строк. Код разбит на четыре модуля: `data.py` читает файл и делит выборку, `features.py` добавляет временные признаки, `model.py` оборачивает `GradientBoostingRegressor`, а `main.py` собирает всё в один сценарий. Ещё есть один тест в `tests/test_data.py`, файл `requirements` и `README`. Я прошёлся по каждому файлу по чек-листу модуля: синтаксис, стиль, документация, работа с данными, архитектура модели и метрики.

Что я нашёл в первую очередь

Самое неприятное открытие: пайплайн вообще не запускается. В `main.py` путь к данным указан как `data/main.csv`, а файл в репозитории называется `uber.csv`. То есть код в текущем виде падает на первой же строке сценария.

Дальше воспроизводимость. Нигде не зафиксирован `random_state`, ни в `train_test_split`, ни в самой модели. Я запускал пайплайн несколько раз и каждый раз получал другое разбиение и другой результат, так что сравнивать эксперименты между собой в таком виде нельзя.

Потом я посмотрел на сами данные, и оказалось, что их никто не чистит. В датасете есть пропуски в координатах высадки, отрицательные цены поездок, поездки с нулём пассажиров и одна с 208 пассажирами, плюс около трёх тысяч строк с нулевыми координатами. Я решил проверить, насколько это влияет на модель: убрал аномалии, а это всего примерно 2,5 процента строк, перетренировал модель, и у меня получилось поднять R^2 на тесте с 0.66 до 0.76. То есть простая очистка данных даёт заметный прирост качества, а модуля предобработки в проекте нет вообще.

И ещё одна вещь, которую стоит исправить сразу: весь код в `main.py` написан на уровне модуля, без функции `main` и без проверки `if __name__ == "__main__"`. Любой импорт этого файла, например из теста, тут же запустит обучение модели.

Метрики, модель и зависимости

С метриками сейчас скромно: считается только R^2 через `model.model.score` и печатается с округлением до двух знаков, чего мало, если захочется сравнивать близкие эксперименты. Метод `predict` нигде не вызывается, это по сути мёртвый код. Метрики никуда не сохраняются, оценки на тренировочной выборке нет, так что переобучение заметить

нечем. Для задачи про стоимость поездки я бы ещё добавил MAE и RMSE, они понятнее читаются в долларах.

Обученная модель тоже никуда не сохраняется, ни в `pkl`, ни в `joblib`, и никакого трекинга экспериментов нет. Гиперпараметры при этом дефолтные, подбора не было. Класс `TaxiFareModel` меня отдельно смутил: он ничего не добавляет к `GradientBoostingRegressor`, а обращаться к нему приходится через неуклюжее `model.model`. Либо класс должен нести реальную ответственность, либо проще работать с `sklearn` напрямую.

С зависимостями я тоже повозился. Файл `requirements` лежит без расширения, версии в нём из 2021 года, транзитивные зависимости не зафиксированы, а `sklearn 0.24` при этом несовместим с `numpy 2.x`, который поставится по умолчанию. `Lock`-файла нет, версия питона нигде не указана, так что точно воспроизвести окружение не выйдет. Я бы перевёл проект на `uv` с `pyproject.toml`.

Тесты, стиль и документация

Тест на весь проект один, и он проверяет только, что `CSV` загружается. Вся логика преобразований, то есть `add_time_features`, `split_data`, `fit` и `predict`, не покрыта. При этом `add_time_features` молча изменяет переданный `DataFrame` вместо того, чтобы работать с копией, и без теста такое легко пропустить. Валидации данных тоже нет никакой: ни проверки схемы, ни диапазонов значений.

Линтеры в проекте не настроены. Я запустил `ruff` локально: `check` нашёл 9 ошибок, а `format` переписал бы все файлы в `src`, там одинарные кавычки, нет пустых строк между функциями и переводов строки в конце файлов. Типизации нет ни в одной сигнатуре, докстрингов тоже. `Pydoclint` я поставил и запустил сам, он выдал 14 замечаний. В комментариях к PR я предложил докстринги в гугловском стиле для всех функций и конфиги для `ruff` и `pydoclint`.

`README` сейчас состоит из заголовка и строки «Run main.py». Нет ни установки, ни описания данных, ни примеров использования. Я написал в PR полный вариант `README` с установкой через `uv`, описанием всех колонок датасета и списком известных проблем.

Что я рекомендую сделать

Начать стоит с того, что чинится за пять минут: исправить путь к данным и обернуть сценарий в функцию `main`. Следующим шагом я бы зафиксировал `random_state` в разбиении и в модели, без этого все дальнейшие эксперименты бессмысленно сравнивать. Потом вынести предобработку данных в отдельный модуль, убрать пропуски и аномалии и покрыть эту логику тестами, тут же лежит самый большой прирост качества. Дальше перевести окружение на `uv` с зафиксированными версиями, настроить `ruff` и `pydoclint`, добавить типизацию и докстринги. И в конце наладить сохранение модели и метрик в файлы рядом с моделью, а в перспективе подключить `mlflow` и подобрать гиперпараметры через `GridSearch` или `optuna`.

Все замечания из этого отчёта я продублировал комментариями к конкретным строкам в PR, там же есть примеры исправлений.